CS3212 Algorithms

Aya Zirikly
11/03/2026

Quiz 4 11/05

Module 3: Optimization & Sequence Modeling

Minimum Spanning Trees: Kruskal's Algorithm & Union-Find

Minimum Spanning Tree (MST) Definition

Given a connected, undirected, edge-weighted graph G = (V, E) with weight function w(e):

- **Spanning Tree:** A connected, acyclic subgraph T = (V, E') containing all vertices V with exactly |V| - 1 edges.
- **Minimum Spanning Tree:** A spanning tree T that minimizes the total edge weight:

Weight(T) = Sum of w(e) for all edges e in T

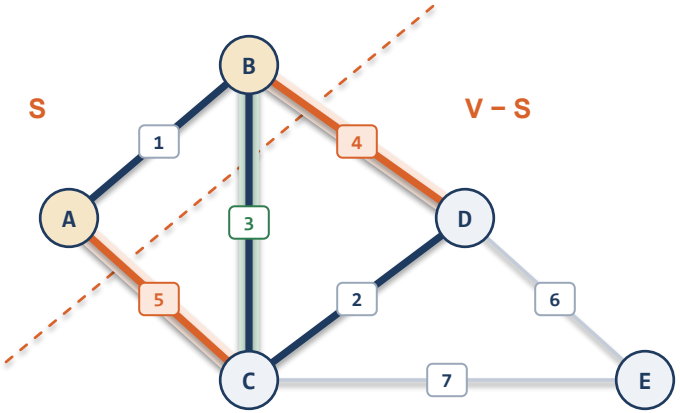
Real-World Applications: fiber-optic network design, cluster analysis, image segmentation.

The Greedy Algorithm Paradigm

- **Greedy Strategy:** Makes a series of locally optimal choices at each step without ever backtracking or revising past choices.
- **Core Goal:** prove that these particular locally optimal choices lead to a global optimum.
- **Why we need a proof:** greedy can fail. With coins {1, 3, 4}, make 6: greedy takes 4 + 1 + 1 (3 coins), but 3 + 3 uses only 2.

The Cut Property & Safe Edge Invariant

Cut: a partition of V into two disjoint sets S and V − S.
Crossing edge: one endpoint in S, the other in V − S.
Respects A: a cut respects an edge set A if no edge of A crosses it.
Light edge: a minimum-weight edge crossing the cut.
Cut Property Theorem: Let A be contained in some MST, and let (S, V − S) be any cut that respects A. If e is a light edge crossing the cut, then **A ∪ {e} is also contained in some MST** (e is safe for A).
Special/base case A = ∅: the lightest edge across any cut belongs to some MST.



In our graph, at step 3 of the trace

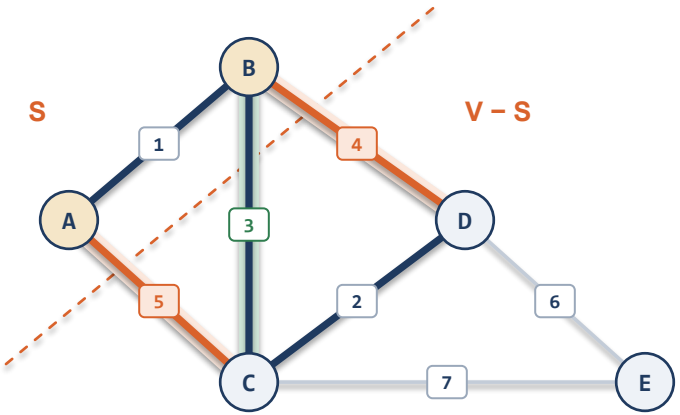
A = {(A, B), (C, D)}, the two edges already accepted.

S = {A, B}. No edge of A crosses, so the cut respects A.

Crossing edges: (B, C) 3, (B, D) 4, (A, C) 5.

Light edge (B, C) is safe, and it is exactly what Kruskal adds next.

The Cut Property & Safe Edge Invariant



Doesn’t respect A: S = {A, C}, V − S = {B, D, E}.

Edge (A,B) crosses the cut because A is in S and B is not, and (C,D) crosses for the same reason.

Cut Property Proof (Exchange Argument)

What we're proving: suppose A is part of some MST T, the cut respects A, and e is a lightest edge crossing the cut. Then A + e is still part of some MST. In other words, adding e is safe.

If T doesn't already use e, we swap e in for a T edge that's at least as heavy, and the result is still an MST.

Cut Property Proof (Exchange Argument)

Setup: $A \subseteq$ some MST T^*, the cut $(S, V - S)$ respects A, and e is a light edge crossing it. If $e \in T^*$, we are done.

Otherwise:

1. Adding e to T^* creates a unique cycle.
2. The cycle crosses the cut at e, so it must cross back at some other edge $e' \in T^*$.
3. $e' \notin A$, because the cut respects A (no edge of A crosses it).
4. e is light, so $w(e) \leq w(e')$. Then $T' = T^* - \{e'\} + \{e\}$ is a spanning tree with $w(T') \leq w(T^*)$.
5. So T' is an MST, and it contains $A \cup \{e\}$, since the only edge removed was $e' \notin A$. Hence e is safe for A.

Kruskal's Algorithm Mechanics & Execution Trace

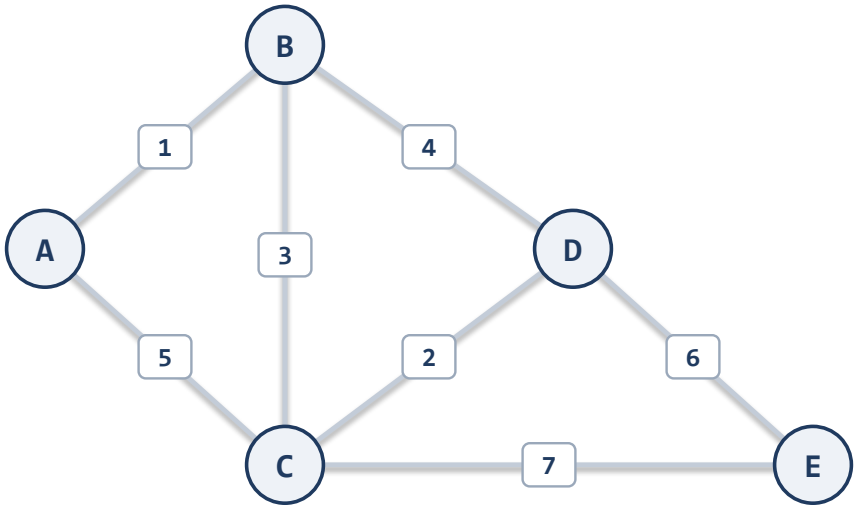
High-Level Strategy

- Start with a spanning forest where every vertex is in its own isolated component (no edges).
- Sort all edges in non-decreasing order of weight: w(e1) <= w(e2) <= ... <= w(em).
- Iterate through the sorted edges: if adding an edge connects two previously disjoint components (does not form a cycle), accept it into the MST.
- Stop when |V| - 1 edges have been accepted.

Trace Example

Graph Parameters: Vertices V = {A, B, C, D, E} (|V| = 5, target edges = 4).

Sorted Edge List: `(A, B): 1`, `(C, D): 2`, `(B, C): 3`, `(B, D): 4`, `(A, C): 5`, `(D, E): 6`, `(C, E): 7`.



Trace at the Set Level

Step	Edge Examined	Weight	Component Check (Find(u) != Find(v))	Decision	Disjoint Set State	Edges Picked	Total Weight
0	-	-	Initial state	-	{A}, {B}, {C}, {D}, {E}	0	0
1	(A, B)	1	Find(A) != Find(B)	ACCEPT	{A, B}, {C}, {D}, {E}	1	1
2	(C, D)	2	Find(C) != Find(D)	ACCEPT	{A, B}, {C, D}, {E}	2	3
3	(B, C)	3	Find(B) != Find(C)	ACCEPT	{A, B, C, D}, {E}	3	6
4	(B, D)	4	Find(B) == Find(D)	REJECT (Cycle)	{A, B, C, D}, {E}	3	6
5	(A, C)	5	Find(A) == Find(C)	REJECT (Cycle)	{A, B, C, D}, {E}	3	6
6	(D, E)	6	Find(D) != Find(E)	ACCEPT	{A, B, C, D, E}	4 (Done)	12

Next: how do we answer "Find(u) = Find(v) is it a cycle?" fast, without a DFS for every edge?

The Disjoint Set Data Structure

Tracks a partition of a set into disjoint components using tree structures:

- `Make-Set(x)`: Creates a single-element set with representative `x` (`parent[x] = x`, `rank[x] = 0`).
- `Find(x)`: Traverses parent pointers up to find the root representative of `x`'s set.
- `Union(x, y)`: Merges the sets containing `x` and `y`.

Kruskal's Algorithm Pseudocode

```
ALGORITHM Kruskal(G = (V, E), w):
    T = Empty Set

    // Step 1: Initialize disjoint sets for each vertex
    FOR EACH vertex v in V:
        Make-Set(v)

    // Step 2: Sort all edges by weight
    Sort edges E in non-decreasing order by weight w

    // Step 3: Greedily pick edges
    FOR EACH edge (u, v) in sorted E:
        IF Find-Set(u) != Find-Set(v):
            T = T UNION {(u, v)}
            Union(u, v)
            IF |T| == |V| - 1:
                BREAK

    RETURN T
```

cycle check

merge

Disjoint Set Union (DSU / Union-Find) Mechanics

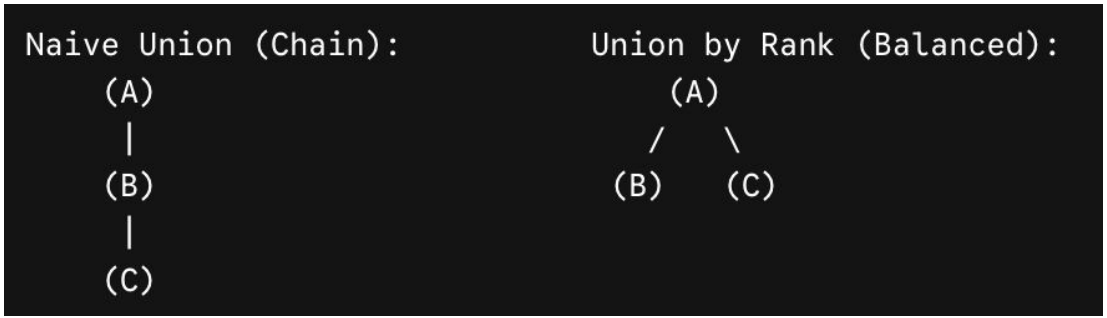
Why Naive Cycle Detection Fails

- Checking for cycles using DFS or BFS takes O(V) time per edge.
- Over |E| edge checks, naive Kruskal would take O(E * V) time, which is too slow for large graphs.

Optimization 1: Union by Rank / Size

Problem: Unbalanced merges can create linear chains of depth O(V), making `Find(x)` take O(V) time.

Solution: Always attach the shorter tree (smaller rank) under the root of the taller tree (larger rank).



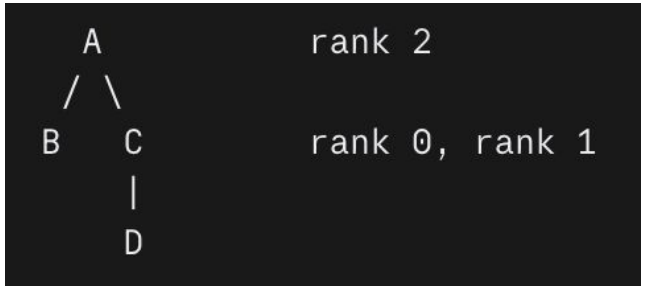
A[0] B[0] C[0] D[0]

A[1] C[0] D[0]
|
B[0]

A[1] C[1]
| |
B[0] D[0]

A[2]
/ \
B[0] C[1]
|
D[0]

A, B, C, D
With union by rank, 4 nodes can be at most 2 levels tall ($\log_2 4 = 2$), so a chain 3 levels tall can't happen. The tallest tree you can get looks like this:



1. Union(A,B): both have rank 0, a tie, so B goes under A and A becomes rank 1.
2. Union(C,D): another tie, so D goes under C and C becomes rank 1.
3. Union(A,C): both have rank 1, a tie, so C goes under A and A becomes rank 2.

Optimization 2: Path Compression



Strategy: During a `Find(x)` query, update the parent pointer of every visited node along the path to point **directly to the root**.

Find(D) walks D → C → A, and on the way back it points every node it passed directly at the root. That means D now hangs straight off A, while C was already a child of A and stays there. Every node is now one step from the root. Union by rank keeps trees short to begin with, and path compression flattens them further each time you call Find.

```
FUNCTION Find(x):
    IF parent[x] != x:
        parent[x] = Find(parent[x])  // Path compression
    RETURN parent[x]

PROCEDURE Union(x, y):
    rootX = Find(x)
    rootY = Find(y)

    IF rootX != rootY:
        // Union by Rank
        IF rank[rootX] < rank[rootY]:
            parent[rootX] = rootY
        ELSE IF rank[rootX] > rank[rootY]:
            parent[rootY] = rootX
        ELSE:
            parent[rootY] = rootX
            rank[rootX] = rank[rootX] + 1
```

Tie rule used here:

on equal ranks, rootY goes under rootX and rank[rootX] grows by 1. The trace uses the same rule.

Textbook note:

CLRS's LINK does the opposite (x goes under y). Both are correct, but parent arrays can differ from the textbook's.

Small saving:

Kruskal already computed both roots in its cycle test, so Union can take the roots directly instead of calling Find again.

Union-find recap, before the trace

`MAKE-SET(x)`	parent[x] = x, rank[x] = 0
`FIND(x)`	follow parent pointers up to the root
`UNION(x, y)`	link the root of one tree under the other

Union by rank

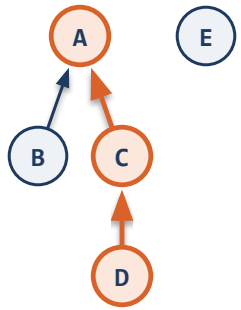
Put the root with smaller rank under the larger one.
On a tie, the second root goes under the first, whose rank grows by 1.

Path compression

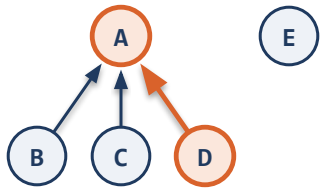
After FIND(x) reaches the root, point every node on the path straight at the root. Later FINDs take one hop.

Both together: m operations on n elements cost O(m α(n)), and α(n) ≤ 4 for any input that fits in our universe.

Coming up in the trace: FIND(D) at step 4



Before: D → C → A (2 hops)

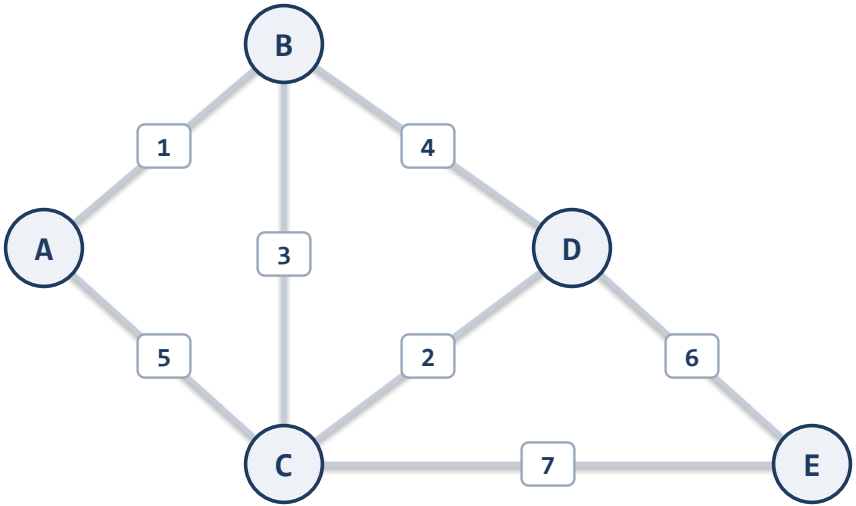


After: D → A (1 hop)

Rank is an upper bound on height. Compression never updates it, so rank[C] stays 1 even though C now has no children.

Kruskal trace: initial state

Graph G, tree edges T in bold



T = ∅ w(T) = 0 |T| = 0 / 4

Sorted edges

(A, B)	1
(C, D)	2
(B, C)	3
(B, D)	4
(A, C)	5
(D, E)	6
(C, E)	7

Union-find state

	A	B	C	D	E
parent	A	B	C	D	E
rank	0	0	0	0	0

Forest (arrows point to parent)



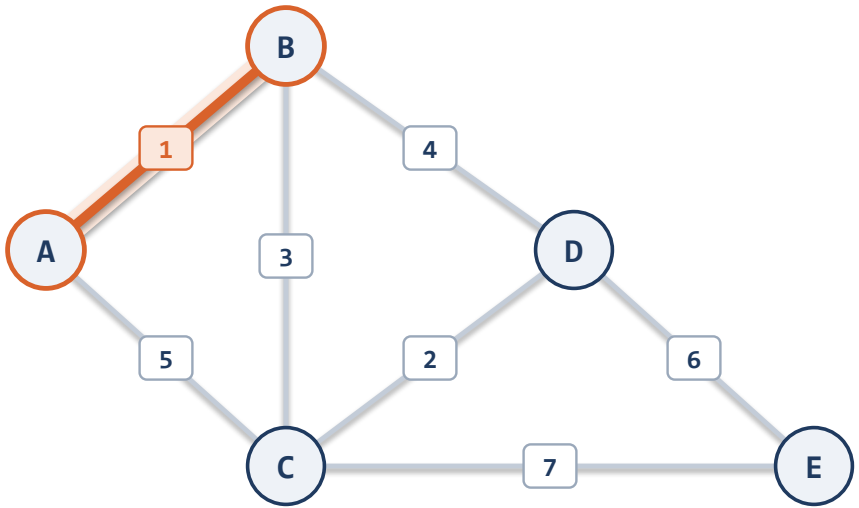
MAKE-SET for every vertex: five one-vertex trees, T = ∅.

parent[v] = v and rank[v] = 0 for all v. The edges are already sorted by weight.

Click → (or press the arrow key) to consider the lightest edge, (A, B) with weight 1.

Kruskal trace, step 1 of 6: edge (A, B), w = 1

Graph G, tree edges T in bold



Sorted edges

(A, B)	1	→ now
(C, D)	2	
(B, C)	3	
(B, D)	4	
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	B	C	D	E
rank	0	0	0	0	0

Forest (arrows point to parent)



T = ∅ w(T) = 0 |T| = 0 / 4

Is (A, B) safe? Compare FIND(A) and FIND(B).

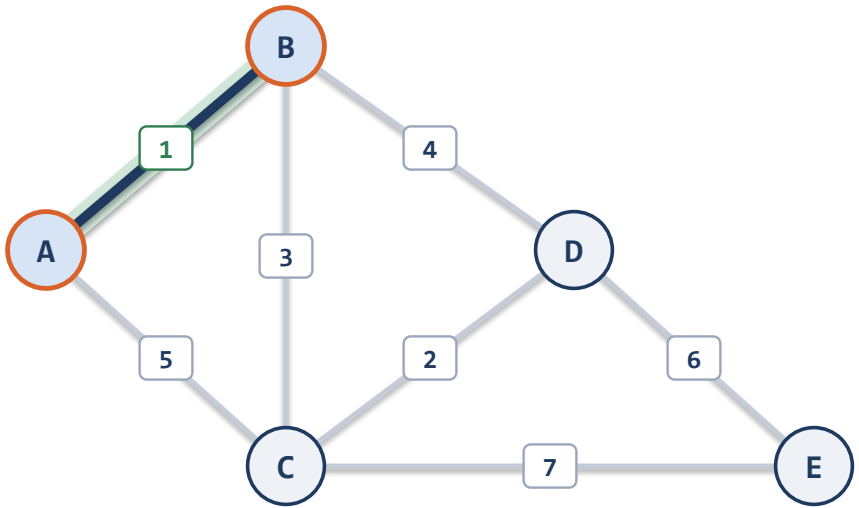
FIND(A) = A (A) FIND(B) = B (B)

Same root means the endpoints are already connected.

✓ Accept ✕ Reject

Kruskal trace, step 1 of 6: edge (A, B), w = 1

Graph G, tree edges T in bold



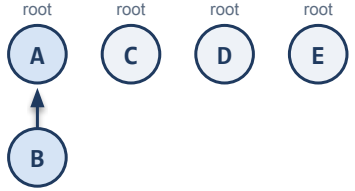
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	
(B, C)	3	
(B, D)	4	
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	C	D	E
rank	1	0	0	0	0

Forest (arrows point to parent)



`T = {(A,B)}` `w(T) = 1` `|T| = 1 / 4`

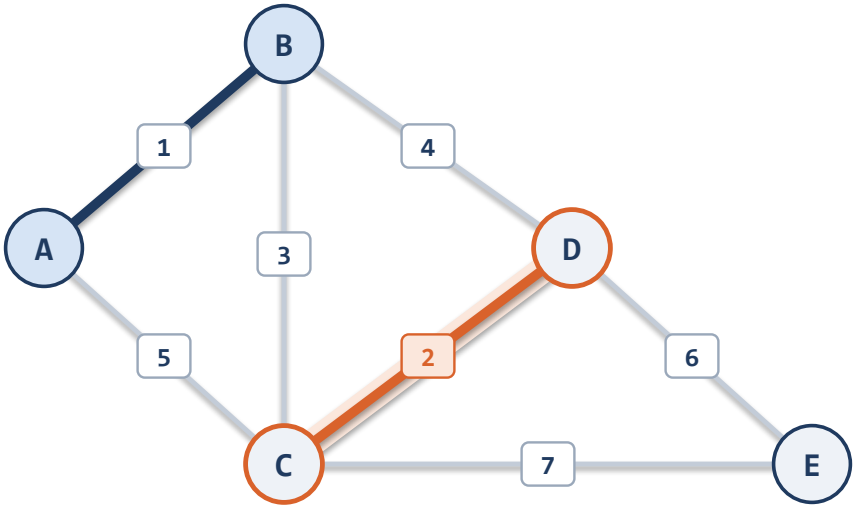
Accept (A, B): roots A ≠ B, so it joins two different trees.

UNION(A, B): ranks tie (0 = 0), so parent[B] ← A and rank[A] becomes 1.

T gains weight 1.

Kruskal trace, step 2 of 6: edge (C, D), w = 2

Graph G, tree edges T in bold



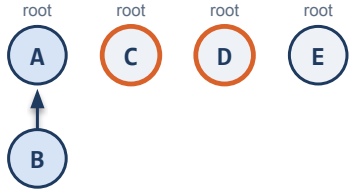
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	→ now
(B, C)	3	
(B, D)	4	
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	C	D	E
rank	1	0	0	0	0

Forest (arrows point to parent)



T = {(A,B)} w(T) = 1 |T| = 1 / 4

Is (C, D) safe? Compare FIND(C) and FIND(D).

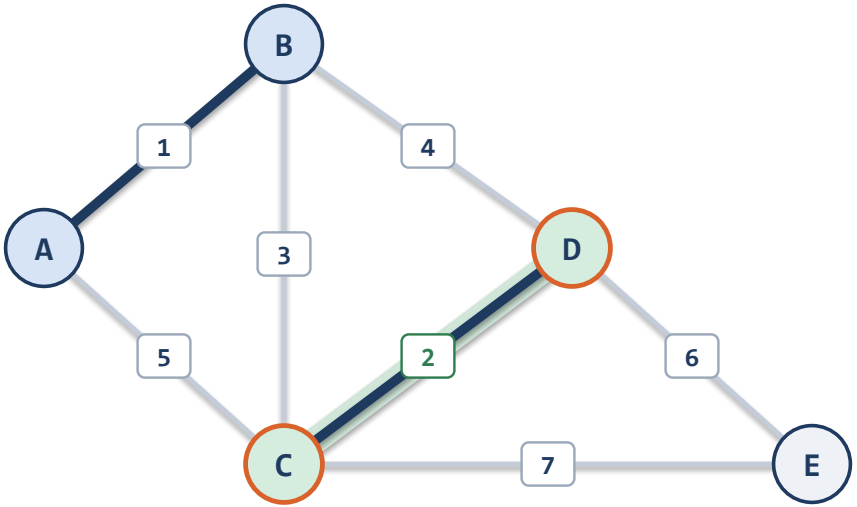
FIND(C) = C (C) FIND(D) = D (D)

Same root means the endpoints are already connected.

✓ Accept ✕ Reject

Kruskal trace, step 2 of 6: edge (C, D), w = 2

Graph G, tree edges T in bold



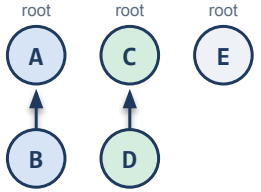
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	
(B, D)	4	
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	C	C	E
rank	1	0	1	0	0

Forest (arrows point to parent)



T = {(A,B), (C,D)} w(T) = 3 |T| = 2 / 4

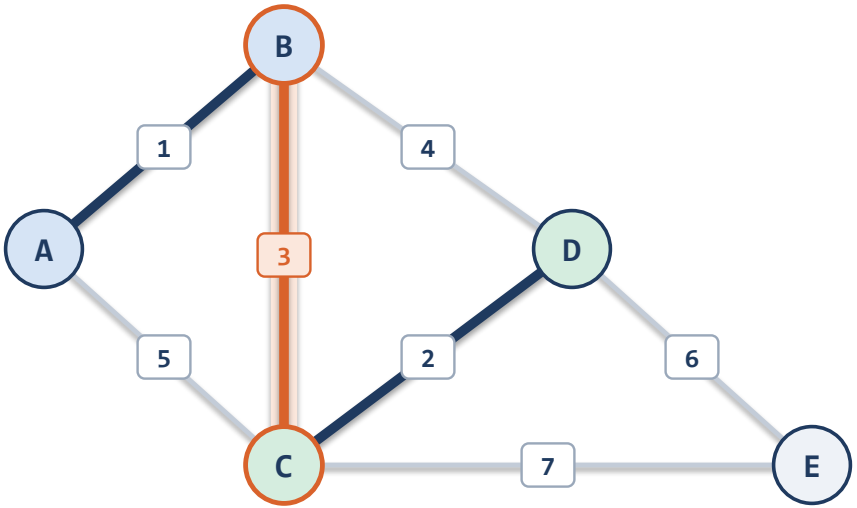
Accept (C, D): roots C ≠ D, so it joins two different trees.

UNION(C, D): ranks tie (0 = 0), so parent[D] ← C and rank[C] becomes 1.

T gains weight 2.

Kruskal trace, step 3 of 6: edge (B, C), w = 3

Graph G, tree edges T in bold



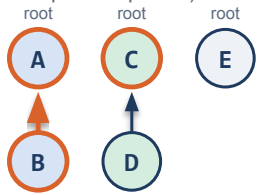
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	→ now
(B, D)	4	
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	C	C	E
rank	1	0	1	0	0

Forest (arrows point to parent)



T = {(A,B), (C,D)} w(T) = 3 |T| = 2 / 4

Is (B, C) safe? Compare FIND(B) and FIND(C).

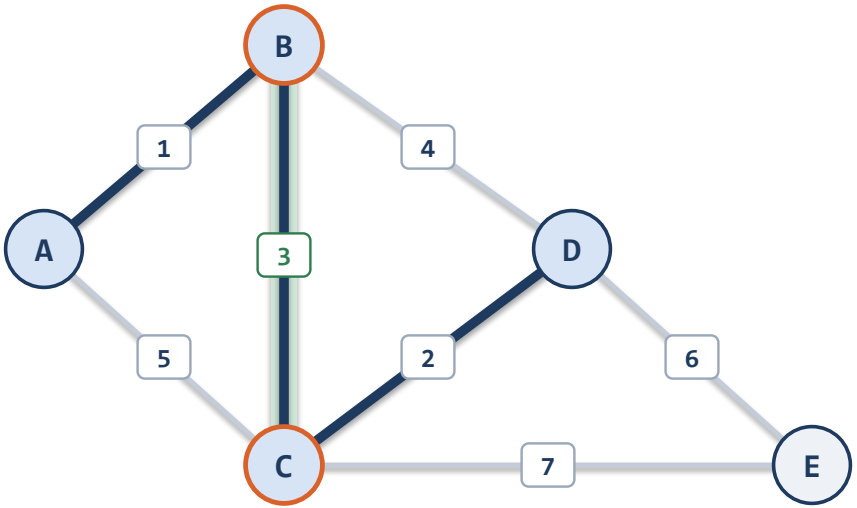
FIND(B) = A (B → A) FIND(C) = C (C)

Same root means the endpoints are already connected.

✓ Accept ✕ Reject

Kruskal trace, step 3 of 6: edge (B, C), w = 3

Graph G, tree edges T in bold



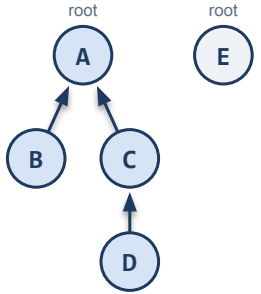
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	C	E
rank	2	0	1	0	0

Forest (arrows point to parent)



`T = {(A,B), (C,D), (B,C)}` `w(T) = 6` `|T| = 3 / 4`

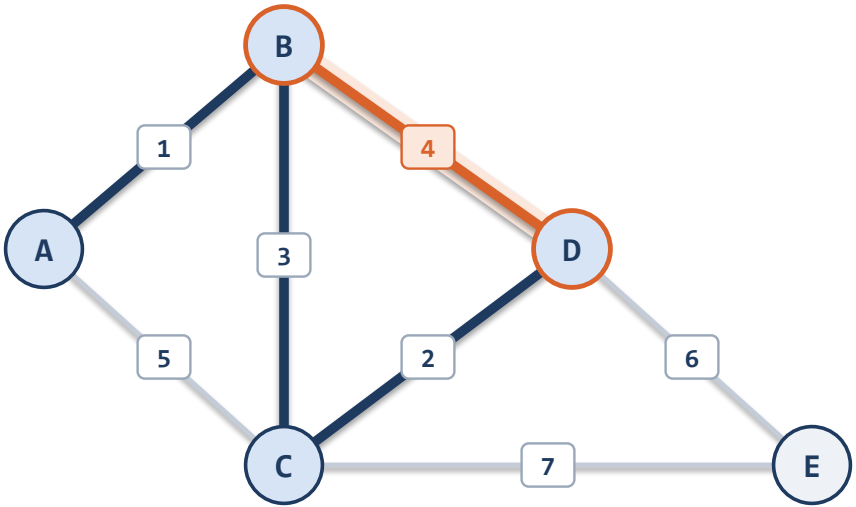
Accept (B, C): roots A ≠ C, so it joins two different trees.

UNION(B, C): ranks tie (1 = 1), so parent[C] ← A and rank[A] becomes 2.

T gains weight 3.

Kruskal trace, step 4 of 6: edge (B, D), w = 4

Graph G, tree edges T in bold



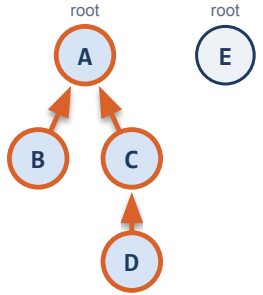
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	→ now
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	C	E
rank	2	0	1	0	0

Forest (arrows point to parent)



T = {(A,B), (C,D), (B,C)} w(T) = 6 |T| = 3 / 4

Is (B, D) safe? Compare FIND(B) and FIND(D).

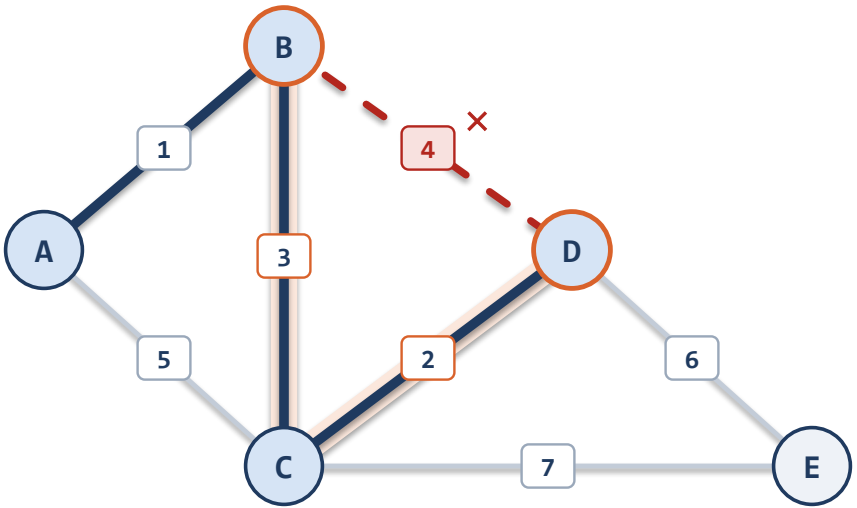
FIND(B) = A (B → A) FIND(D) = A (D → C → A)

Same root means the endpoints are already connected.

✓ Accept ✕ Reject

Kruskal trace, step 4 of 6: edge (B, D), w = 4

Graph G, tree edges T in bold



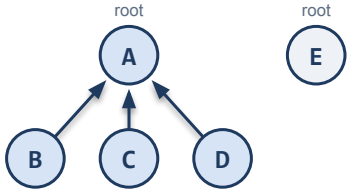
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	A	E
rank	2	0	1	0	0

Forest (arrows point to parent)



`T = {(A,B), (C,D), (B,C)}` `w(T) = 6` `|T| = 3 / 4`

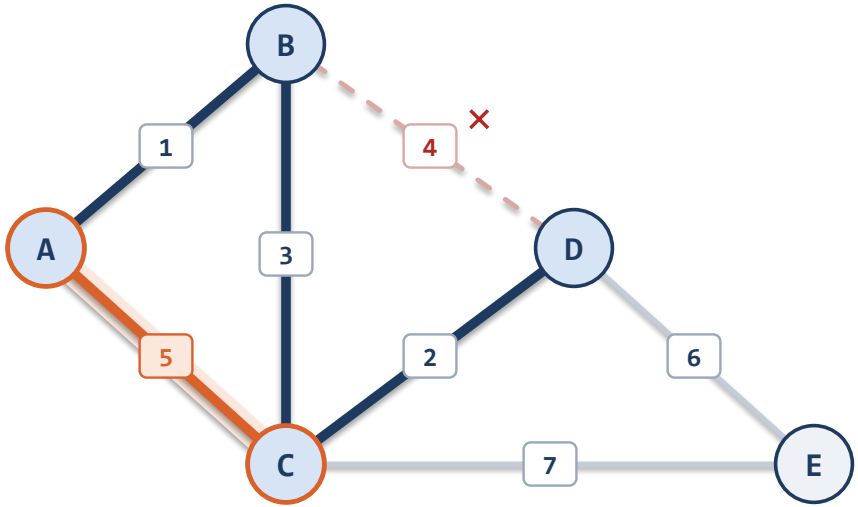
Reject (B, D): both roots are A, so it would close a cycle.

B and D are already joined in T by B – C – D (highlighted). Adding (B, D) makes the cycle B – C – D – B.

No union. Path compression: FIND walked D → C → A, then set parent[D] ← A.

Kruskal trace, step 5 of 6: edge (A, C), w = 5

Graph G, tree edges T in bold



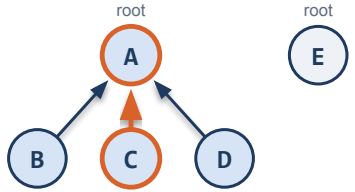
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	→ now
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	A	E
rank	2	0	1	0	0

Forest (arrows point to parent)



T = {(A,B), (C,D), (B,C)} w(T) = 6 |T| = 3 / 4

Is (A, C) safe? Compare FIND(A) and FIND(C).

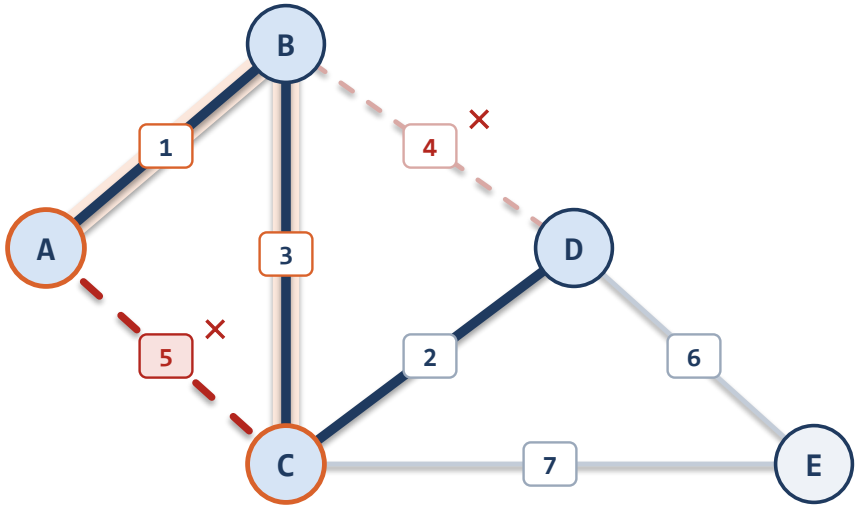
FIND(A) = A (A) FIND(C) = A (C → A)

Same root means the endpoints are already connected.

✓ Accept ✕ Reject

Kruskal trace, step 5 of 6: edge (A, C), w = 5

Graph G, tree edges T in bold



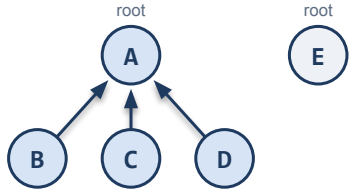
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	✕ cycle
(D, E)	6	
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	A	E
rank	2	0	1	0	0

Forest (arrows point to parent)



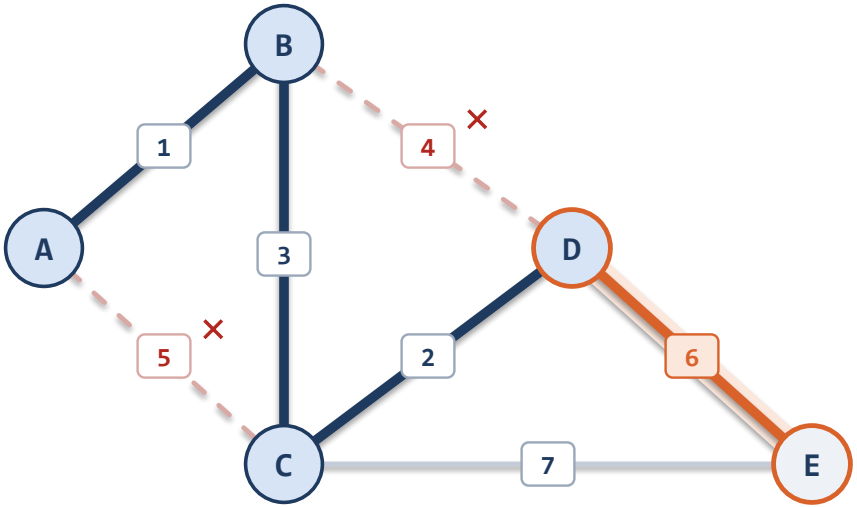
T = {(A,B), (C,D), (B,C)} w(T) = 6 |T| = 3 / 4

Reject (A, C): both roots are A, so it would close a cycle.

A and C are already joined in T by A – B – C (highlighted). Adding (A, C) makes the cycle A – B – C – A.
No union happens; the union-find state does not change.

Kruskal trace, step 6 of 6: edge (D, E), w = 6

Graph G, tree edges T in bold



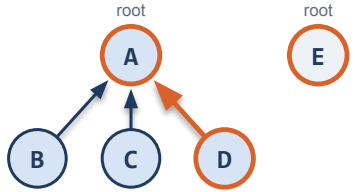
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	✕ cycle
(D, E)	6	→ now
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	A	E
rank	2	0	1	0	0

Forest (arrows point to parent)



T = {(A,B), (C,D), (B,C)} w(T) = 6 |T| = 3 / 4

Is (D, E) safe? Compare FIND(D) and FIND(E).

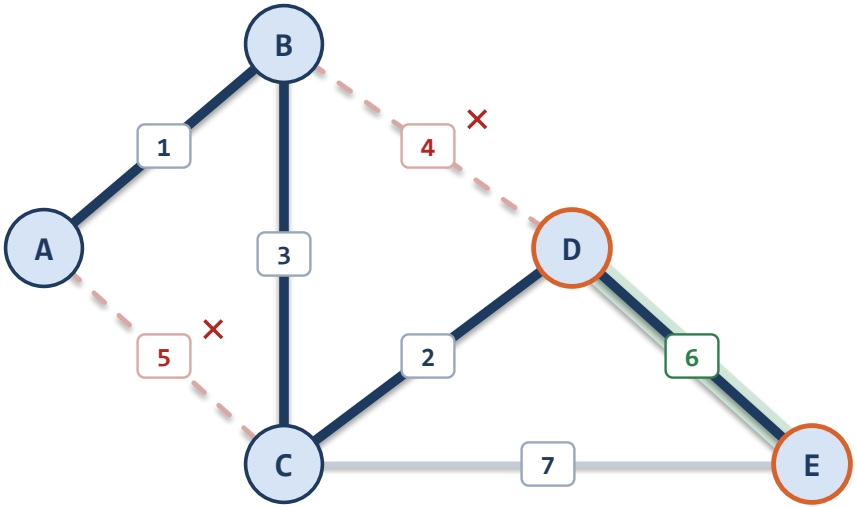
FIND(D) = A (D → A) FIND(E) = E (E)

Same root means the endpoints are already connected.

✓ Accept ✕ Reject

Kruskal trace, step 6 of 6: edge (D, E), w = 6

Graph G, tree edges T in bold



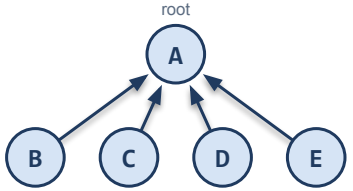
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	✕ cycle
(D, E)	6	✓ in T
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	A	A
rank	2	0	1	0	0

Forest (arrows point to parent)



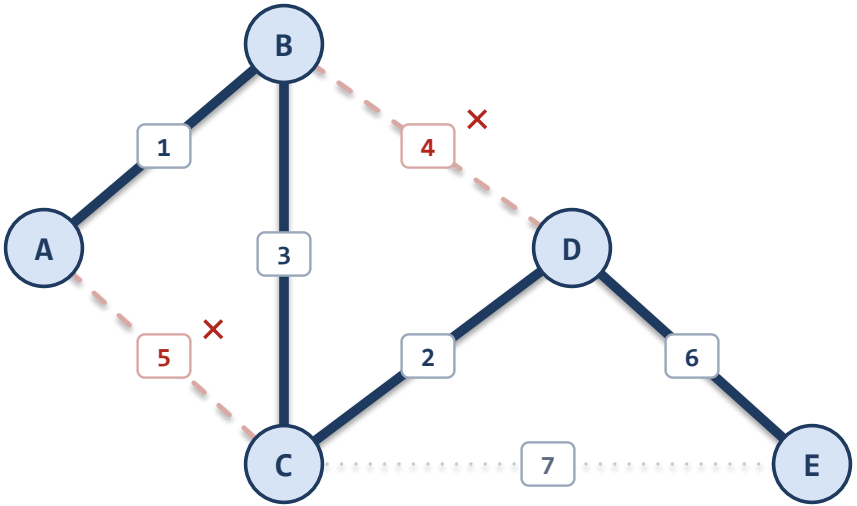
`T = {(A,B), (C,D), (B,C), (D,E)}` `w(T) = 12` `|T| = 4 / 4`

Accept (D, E): roots A ≠ E, so it joins two different trees.

UNION(D, E): rank[A] = 2 > rank[E] = 0, so the shorter tree goes under: parent[E] ← A.
T gains weight 6.

Kruskal trace: finished

Graph G, tree edges T in bold



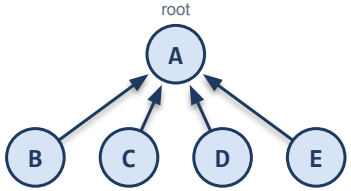
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	✕ cycle
(D, E)	6	✓ in T
(C, E)	7	not needed

Union-find state

	A	B	C	D	E
parent	A	A	A	A	A
rank	2	0	1	0	0

Forest (arrows point to parent)



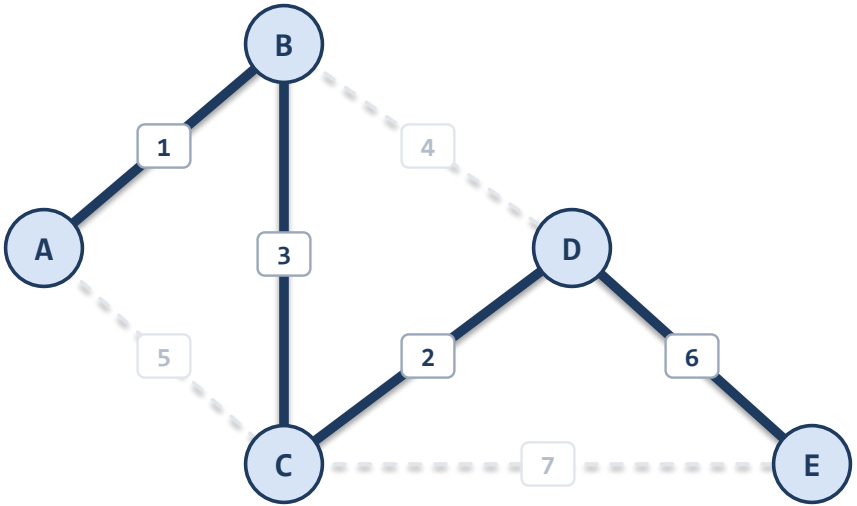
`T = {(A,B), (C,D), (B,C), (D,E)}` `w(T) = 12` `|T| = 4 / 4`

Done: |T| = 4 = |V| − 1, so the loop stops. MST weight = 1 + 2 + 3 + 6 = 12.

(C, E): 7 is never examined. It would have been rejected anyway: FIND(C) = FIND(E) = A.

Every vertex now has root A, so the forest is a single spanning tree.

Result: the MST has weight 12



Every edge, and what happened to it

(A, B)	1	✓ in T	joins {A} and {B}
(C, D)	2	✓ in T	joins {C} and {D}
(B, C)	3	✓ in T	joins {A,B} and {C,D}
(B, D)	4	✕ rejected	cycle B – C – D
(A, C)	5	✕ rejected	cycle A – B – C
(D, E)	6	✓ in T	joins {A,B,C,D} and {E}
(C, E)	7	– skipped	never examined

$$w(T) = 1 + 2 + 3 + 6 = 12$$

Cut check for vertex E: the cut ({E}, {A, B, C, D}) is crossed by (D, E): 6 and (C, E): 7. The lightest crossing edge, (D, E), must be in the MST, and it is.

All seven weights are distinct, so this MST is unique.

Running time

Step	Count	Cost
MAKE-SET	|V|	`O(V)`
Sort the edges	once	`O(E log E)`
FIND	2 per edge examined	`O(E α(V)) total`
UNION	|V| − 1	`(inside the line above)`
Total		`O(E log E) = O(E log V)`

In our example

edges examined	`6 of 7`
FIND calls	`12`
UNION calls	`4`
edges rejected	`2`
longest FIND path	`2 hops`
after compression	`1 hop`

Why log E = O(log V): a simple graph has $E \le V^2$, so $\log E \le 2 \log V$.

The sort dominates. Union-find with both heuristics is almost linear.

Compare: Prim with a binary heap is also O(E log V). Kruskal fits sparse graphs given as an edge list.

Question 1 of 4

If w(B, D) were 2.5 instead of 4, which edge would Kruskal reject first?

- `(B, C)`
- `(B, D)`
- `(A, C)`
- `none`

Click an answer.

Question 2 of 4

Why does Kruskal never look at (C, E): 7?

- |T| = |V| − 1
- largest weight
- C, E separate

Click an answer.

Question 3 of 4

After step 4, what is parent[D]?

D | C | A

Click an answer.

Question 4 of 4

Could a different tie-breaking rule give a different MST here?

yes

no

Click an answer.

Overall Complexity Analysis & Comparison

Amortized Complexity of Union-Find

Combined Optimizations: Using both **Union by Rank** and **Path Compression** guarantees that tree depth almost never grows beyond height 4.
The Runtime Bound: Amortized runtime per operation is `O(alpha(V))`, where `alpha` is the **Inverse Ackermann function**.
What `alpha(V)` Means in Practice:

- `alpha(V)` grows so slowly that `alpha(V) <= 4` for all graph sizes up to `V = 10^600` (which is more than the total number of atoms in the known universe)
- For all real-world computational problems, `alpha(V)` is effectively a constant (`<= 4`).

Key Takeaway: Union-Find performs `Make-Set`, `Find`, and `Union` in **effectively constant O(1) amortized time**.

Kruskal's Runtime Breakdown

Step	Operation	Count	Cost per Operation	Total Time
1	MAKE-SET	V	O(1)	O(V)
2	Sort the edges	once (E edges)		O(E log E)
3	FIND (cycle check)	2E	O(α(V)) amortized	O(E α(V))
4	UNION	V − 1	O(α(V)) amortized	O(V α(V))
	Total			**O(E log E) = O(E log V)**

Kruskal's Runtime Breakdown

Step	Operation	Count	Cost per Operation	Total Time
1	MAKE-SET	V	O(1)	O(V)
2	Sort the edges	once (E edges)		
3	FIND (cycle check)	2E		
4	UNION	V − 1		
	Total			**O(E log E) = O(E log V)**

Kruskal checks every edge (u, v), and each check calls Find twice, once for u and once for v. That's 2 Finds per edge, so 2E Finds in total.

Kruskal's Runtime Breakdown

Step	Operation	Count	Cost per Operation	Total Time
1	MAKE-SET	V	O(1)	O(V)
2	Sort the edges	once (E edges)	n/a	O(E log E)
3	FIND (cycle check)	2E	O(α(V)) amortized	O(E α(V))
4	UNION	V − 1	O(α(V)) amortized	O(V α(V))
	Total			**O(E log E) = O(E log V)**

Total Worst-Case Time Complexity:

Total Time = Initialization + Edge Sorting + Find-Set Queries + Union Operations

Total Time = O(V) + O(E log E) + O(E * alpha(V)) + O(V * alpha(V))

Total Time = O(E log E + E * alpha(V))

Simplification: Since |E| <= |V|^2, we have log E <= log(|V|^2) = 2 log V.

Total Runtime = O(E log V)

Bottleneck: Sorting the edges dominates the total runtime. The Union-Find graph processing phase is nearly linear.

Feature	Optimized Kruskal (with DSU)	Naive Kruskal (without DSU)
Cycle Detection Method	Disjoint Set Union (Find-Set)	Graph Search (DFS / BFS) per edge
Cost per Cycle Check	O(alpha(V)) amortized ~ O(1)	O(V) per edge test
Cycle Check Total Time	O(E * alpha(V))	O(E * V)
Overall Time Complexity	O(E log V)	O(E * V)
Impact on Bottleneck	Sorting edges is the sole bottleneck	Cycle detection overpowers edge sorting

Running time

Go back to our example

Step	Count	Cost
MAKE-SET	|V|	`O(V)`
Sort the edges	once	`O(E log E)`
FIND	2 per edge examined	`O(E α(V)) total`
UNION	|V| − 1	`(inside the line above)`
Total		`O(E log E) = O(E log V)`

In our example

edges examined	`6 of 7`
FIND calls	`12`
UNION calls	`4`
edges rejected	`2`
longest FIND path	`2 hops`
after compression	`1 hop`

Why log E = O(log V): a simple graph has $E \le V^2$, so $\log E \le 2 \log V$.

The sort dominates. Union-find with both heuristics is almost linear.

Compare: Prim with a binary heap is also O(E log V). Kruskal fits sparse graphs given as an edge list.